

Private Network Service Platform

Computer Networks Course Project · Phase 1 Report · Team team1

Project summary	
Private domain	app.team1.test , api.team1.test
Network	4 macOS laptops on one Wi-Fi LAN, 10.7.0.0/19, gateway 10.7.0.1
Stack	dnsmasq · nginx 1.31.6 · OpenSSL local CA · Python REST backends · curl · dig · Wireshark
Evidence captured	4 October 2026
Status	Phase 1 tasks A–G working end to end over HTTPS with certificate verification (no -k)

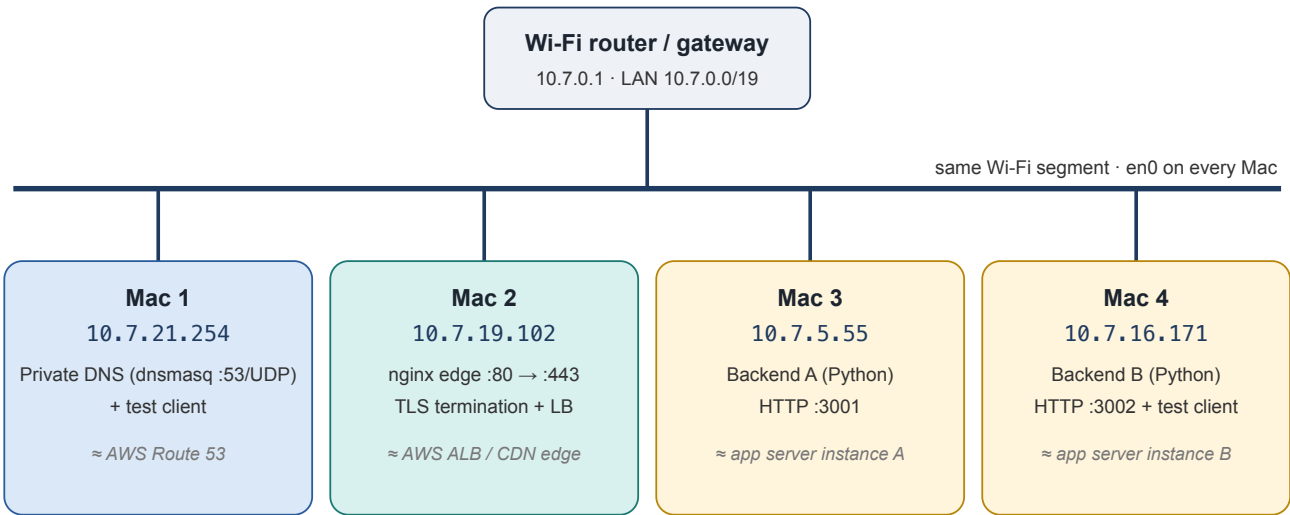
Every terminal output and screenshot in this report is from a real run on the team machines. The two diagrams (topology and request flow) were drawn for this report from the project configuration. Section 12 is the video walkthrough script.

1. Machines, roles and IP inventory

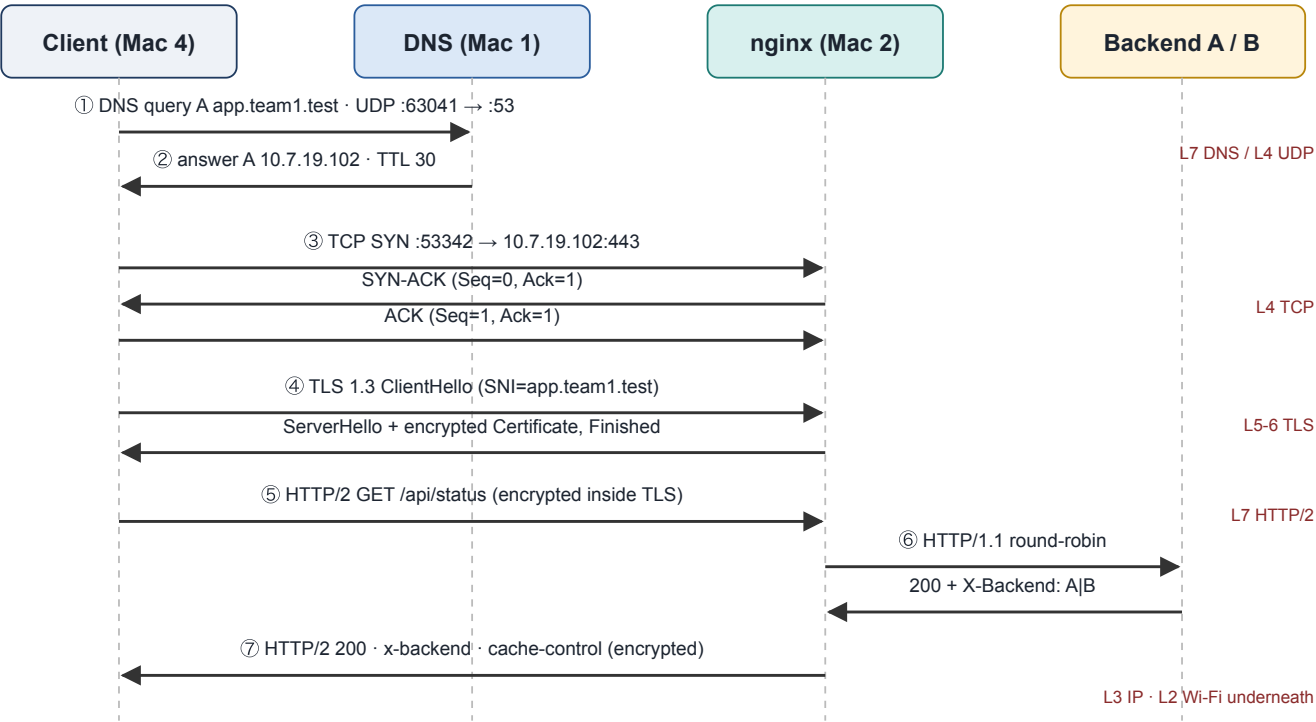
Machine	Role	IP (en0)	Services / ports	Cloud equivalent
Mac 1	Private DNS server + test client	10.7.21.254	dnsmasq UDP/TCP 53	Route 53 (private hosted zone)
Mac 2	Edge reverse proxy, TLS termination, load balancer	10.7.19.102	nginx 80 (→301), 443 (TLS, HTTP/2)	ALB / CDN edge
Mac 3	Backend A	10.7.5.55	Python HTTP 3001	App server instance A
Mac 4	Backend B + test client	10.7.16.171	Python HTTP 3002	App server instance B

Subnet mask 255.255.224.0 (/19), default gateway 10.7.0.1, active interface en0 on every Mac. All IPs live in one file, `config.env`; `./render.sh` turns the templates into the real dnsmasq, nginx and firewall configs, so all four Macs stay consistent.

Network topology



Request flow — one request through every layer



Step	Protocol	OSI / TCP-IP layer	Ports	Proof in this report
Name → IP	DNS	Application (over UDP)	ephemeral → 53/UDP	dig (§3), Wireshark frames 324/325 (§8)
Connection	TCP 3-way handshake	Transport	ephemeral 53342 → 443/TCP	Wireshark 56632–56634 (§8)
Security	TLS 1.3	Session / presentation	443	curl -v, Wireshark 56635+ (§6, §8)
Request	HTTP/2 client↔edge, HTTP/1.1 edge↔backend	Application	443 → 3001 / 3002	X-Backend headers (§5)
Delivery	IPv4 · Wi-Fi 802.11	Network · Link	—	ping, MAC addresses in frames

2. Task A — Private LAN

All four Macs joined the same Wi-Fi. `scripts/netinfo.sh` records IP, mask, gateway, interface and MAC address; `scripts/pingall.sh` pings every team machine.

```
● (base) arun.@Aruns-MacBook-Pro-2 MAC2 % ./render.sh
scripts/netinfo.sh | tee evidence/phase1/A1-netinfo-$(hostname -s).txt
scripts/pingall.sh | tee evidence/phase1/A2-pingall-$(hostname -s).txt
rendered build/dns/dnsmasq.conf
rendered build/dns/team.hosts
rendered build/nginx/edge-phase1.conf
rendered build/nginx/edge-phase2.conf
rendered build/firewall/backend-pf.conf

== app.team1.test - interface en0 ==
IPv4 address : 10.7.19.102
Subnet mask  : 255.255.224.0
Prefix       : 0xffffe000 (hex mask)
Gateway      : 10.7.0.1
Default if   : en0
MAC address   : de:18:32:22:fd:bd
DNS servers   : 10.7.21.254
Mac1 10.7.21.254 OK round-trip min/avg/max/stddev = 6.192/9.671/11.545/2.463 ms
Mac2 10.7.19.102 OK round-trip min/avg/max/stddev = 0.202/0.223/0.252/0.021 ms
Mac3 10.7.5.55 OK round-trip min/avg/max/stddev = 6.299/39.014/100.921/43.799 ms
Mac4 10.7.16.171 OK round-trip min/avg/max/stddev = 80.773/169.269/222.328/62.988 ms
```

Mac 2: `./render.sh`, then `netinfo` (10.7.19.102/19, gw 10.7.0.1, en0, MAC de:18:32:22:fd:bd, DNS 10.7.21.254) and `pingall` — all four Macs OK.

Ping summary

```
Mac1 → Mac2: ping 10.7.19.102 - OK (avg 52.464 ms)
Mac1 → Mac3: ping 10.7.5.55 - OK (avg 6.786 ms)
Mac1 → Mac4: ping 10.7.16.171 - OK (avg 76.950 ms)
Mac2 → Mac1: ping -c 4 10.7.21.254 - 4 packets transmitted, 4 received, 0.0% loss
Mac2 → Mac3: ping -c 4 10.7.5.55 - 4 packets transmitted, 4 received, 0.0% loss
Mac2 → Mac4: ping -c 4 10.7.16.171 - 4 packets transmitted, 4 received, 0.0% loss
Mac3 → Mac4: ping -c 4 10.7.16.171 - not captured yet (run on Mac 3)
```

Every pair measured so far: 0% loss.

Mac 2 → others, ping -c 4

```
== ping -c 4 10.7.21.254
--- 10.7.21.254 ping statistics ---
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 26.751/65.044/109.365/29.899 ms
== ping -c 4 10.7.5.55
--- 10.7.5.55 ping statistics ---
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 7.223/87.722/326.011/137.581 ms
== ping -c 4 10.7.16.171
--- 10.7.16.171 ping statistics ---
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 12.745/19.628/26.711/6.456 ms
```

3. Task B — Private DNS server (Mac 1, dnsmasq)

dnsmasq.conf (Mac 1) — relevant lines

```
port=53
listen-address=127.0.0.1,10.7.21.254
domain-needed
bogus-priv
```

```
no-resolv
server=1.1.1.1
local=/team1.test/
addn-hosts=/opt/homebrew/etc/dnsmasq.d/team.hosts
local-ttl=30
```

team.hosts – project records (loaded with addn-hosts=)

```
10.7.19.102 app.team1.test
10.7.19.102 api.team1.test
10.7.21.254 dns1.team1.test
10.7.5.55 dns2.team1.test
10.7.19.102 edge.team1.test
10.7.5.55 backend-a.team1.test
10.7.16.171 backend-b.team1.test
```

local=/team1.test/ makes Mac 1 the only authority for *.team1.test; everything else is forwarded to 1.1.1.1. listen-address includes the LAN IP so other Macs can query it. Clients were pointed at Mac 1 with scripts/client-dns.sh primary (sets Wi-Fi DNS to 10.7.21.254 and flushes the cache).

dig from a client (Mac 2) – answered by our DNS 10.7.21.254

```
$ dig app.team1.test

; <<>> DiG 9.10.6 <<>> app.team1.test
;; global options: +cmd
;; Got answer:
;; -->HEADER<<-- opcode: QUERY, status: NOERROR, id: 4237
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;app.team1.test.                IN      A

;; ANSWER SECTION:
app.team1.test.                30      IN      A      10.7.19.102

;; Query time: 332 msec
;; SERVER: 10.7.21.254#53(10.7.21.254)
;; WHEN: Sun Oct 04 20:12:28 IST 2026
;; MSG SIZE rcvd: 59
```

dig @8.8.8.8 – NXDOMAIN: the name exists only in our private DNS

```
$ dig @8.8.8.8 app.team1.test

; <<>> DiG 9.10.6 <<>> @8.8.8.8 app.team1.test
; (1 server found)
;; global options: +cmd
;; Got answer:
;; -->HEADER<<-- opcode: QUERY, status: NXDOMAIN, id: 32619
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;app.team1.test.                IN      A

;; AUTHORITY SECTION:
.                86325   IN      SOA     a.root-servers.net. nstld.verisign-grs.com. 2026100400 1800 9
00 604800 86400

;; Query time: 16 msec
;; SERVER: 8.8.8.8#53(8.8.8.8)
;; WHEN: Sun Oct 04 20:12:03 IST 2026
;; MSG SIZE rcvd: 118
```

4. Task C — Two backends

Mac 3 and Mac 4 run the same small Python server (backend/run.sh A|B) bound to 0.0.0.0, so the edge can reach it over the LAN. Endpoints: /, /api/status (JSON with backend id), /api/catalog (cacheable), /health. Every response carries X-Backend.

```
• (base) arun.@Aruns-MacBook-Pro-2 MAC2 % curl -i http://10.7.16.171:3002/health
HTTP/1.1 200 OK
Server: TeamBackend/1.0 Python/3.12.5
Date: Sun, 04 Oct 2026 13:02:49 GMT
X-Backend: B
Content-Type: text/plain
Content-Length: 3

ok
• (base) arun.@Aruns-MacBook-Pro-2 MAC2 % curl -i http://10.7.5.55:3001/health
HTTP/1.1 200 OK
Server: TeamBackend/1.0 Python/3.13.6
Date: Sun, 04 Oct 2026 13:03:23 GMT
X-Backend: A
Content-Type: text/plain
Content-Length: 3

ok
```

From Mac 2 directly to each backend: B on 10.7.16.171:3002 and A on 10.7.5.55:3001 both answer 200 with X-Backend.

5. Task D — nginx reverse proxy and load balancer (Mac 2)

```
/opt/homebrew/etc/nginx/servers/team.conf

# ===== Team team1 edge (Mac 2) - PHASE 1 =====
# Installed as: /opt/homebrew/etc/nginx/servers/team.conf
# (Homebrew's nginx.conf already does `include servers/*;` inside http{})

log_format team_lb '$remote_addr:$remote_port "$request" $status '
                   'upstream=$upstream_addr up_status=$upstream_status '
                   'tls=$ssl_protocol rt=$request_time';

# The pool of backends. Default algorithm = round robin.
# (alternative: add `least_conn;` and explain it in the viva)
upstream team_backends {
    server 10.7.5.55:3001;    # Backend A (Mac 3)
    server 10.7.16.171:3002; # Backend B (Mac 4)
}

# Plain HTTP -> redirect to HTTPS
server {
    listen 80;
    server_name app.team1.test api.team1.test;
    return 301 https://$host:443$request_uri;
}

# HTTPS: TLS terminates HERE; backends receive plain HTTP on the LAN
server {
    listen 443 ssl;
    http2 on;                                # nginx >= 1.25.1
    server_name app.team1.test api.team1.test;

    ssl_certificate      /opt/homebrew/etc/nginx/certs/server.crt;
    ssl_certificate_key  /opt/homebrew/etc/nginx/certs/server.key;
    ssl_protocols        TLSv1.2 TLSv1.3;
```

```

access_log /tmp/nginx-team-access.log team_lb;
error_log /tmp/nginx-team-error.log warn;

add_header X-Edge $hostname always; # which edge answered
add_header X-Upstream $upstream_addr always; # which backend IP:port

location / {
    proxy_pass http://team_backends;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header Connection "";
}
}

```

The upstream `team_backends` pool uses nginx's default **round-robin**. Clients only ever know `app.team1.test` → 10.7.19.102; backend IPs are hidden behind the edge. nginx adds `X-Forwarded-For / -Proto` so backends still see the real client and scheme.

6 consecutive requests – headers

```

$ for i in {1..6}; do curl -s -D - -o /dev/null https://app.team1.test/api/status | grep -i '^x-backend'; done
x-backend: B
x-backend: A
x-backend: B
x-backend: A
x-backend: B
x-backend: A

```

```
for i in {1..6}; do curl -s https://app.team1.test/api/status; echo; done
```

```

$ for i in {1..6}; do curl -s https://app.team1.test/api/status; echo; done
{
  "backend": "B",
  "status": "ok",
  "host": "Keshavs-MacBook-Pro-2.local",
  "port": 3002,
  "uptime_s": 5998.8,
  "tcp_peer_ip": "10.7.19.102",
  "x_forwarded_for": "10.7.19.102",
  "x_forwarded_proto": "https",
  "host_header": "app.team1.test"
}

{
  "backend": "A",
  "status": "ok",
  "host": "Mayank-Thukrans-MacBook-Pro.local",
  "port": 3001,
  "uptime_s": 5964.2,
  "tcp_peer_ip": "10.7.19.102",
  "x_forwarded_for": "10.7.19.102",
  "x_forwarded_proto": "https",
  "host_header": "app.team1.test"
}

{
  "backend": "B",
  "status": "ok",
  "host": "Keshavs-MacBook-Pro-2.local",
  "port": 3002,
  "uptime_s": 5998.9,
  "tcp_peer_ip": "10.7.19.102",
  "x_forwarded_for": "10.7.19.102",
  "x_forwarded_proto": "https",
  "host_header": "app.team1.test"
}

```

```
}

{
  "backend": "A",
  "status": "ok",
  "host": "Mayank-Thukrans-MacBook-Pro.local",
  "port": 3001,
  "uptime_s": 5964.3,
  "tcp_peer_ip": "10.7.19.102",
  "x_forwarded_for": "10.7.19.102",
  "x_forwarded_proto": "https",
  "host_header": "app.team1.test"
}

{
  "backend": "B",
  "status": "ok",
  "host": "Keshavs-MacBook-Pro-2.local",
  "port": 3002,
  "uptime_s": 5999.0,
  "tcp_peer_ip": "10.7.19.102",
  "x_forwarded_for": "10.7.19.102",
  "x_forwarded_proto": "https",
  "host_header": "app.team1.test"
}

{
  "backend": "A",
  "status": "ok",
  "host": "Mayank-Thukrans-MacBook-Pro.local",
  "port": 3001,
  "uptime_s": 5964.4,
  "tcp_peer_ip": "10.7.19.102",
  "x_forwarded_for": "10.7.19.102",
  "x_forwarded_proto": "https",
  "host_header": "app.team1.test"
}
```


6. Task E — HTTPS / TLS

A private CA (team1 Local Root CA) was created with OpenSSL (tls/make-certs.sh) and signs the server certificate for app.team1.test / api.team1.test. nginx terminates TLS on 443. Only the public ca.crt is copied to client Macs and trusted in the System keychain (tls/trust-ca.sh); private keys stay on Mac 2 with mode 600.

Certificates and trust (Mac 2)

```
$ openssl x509 -in tls/out/ca.crt -noout -subject -issuer -dates
subject=CN=team1 Local Root CA, O=CN Project team1
issuer=CN=team1 Local Root CA, O=CN Project team1
notBefore=Oct  4 13:11:02 2026 GMT
notAfter=Jan  6 13:11:02 2029 GMT

$ openssl x509 -in tls/out/server.crt -noout -subject -issuer -dates -ext subjectAltName
subject=CN=app.team1.test
issuer=CN=team1 Local Root CA, O=CN Project team1
notBefore=Oct  4 13:11:02 2026 GMT
notAfter=Nov  5 13:11:02 2027 GMT
X509v3 Subject Alternative Name:
    DNS:app.team1.test, DNS:api.team1.test

$ openssl verify -CAfile tls/out/ca.crt tls/out/server.crt
tls/out/server.crt: OK

$ sudo security add-trusted-cert -d -r trustRoot -k /Library/Keychains/System.keychain tls/out/ca.crt
$ security find-certificate -a -c "team1 Local Root CA" /Library/Keychains/System.keychain
    "alis"<blob>="team1 Local Root CA"
    "labl"<blob>="team1 Local Root CA"

$ ls -l tls/out/server.key tls/out/ca.key          # private keys never leave Mac 2
-rw-----@ 1 arun.  staff  3272 Oct  4 18:41 tls/out/ca.key
-rw-----@ 1 arun.  staff  1704 Oct  4 18:41 tls/out/server.key
```

curl -v https://app.team1.test - no -k, domain name only

```
$ curl -v https://app.team1.test
* Host app.team1.test:443 was resolved.
* IPv6: (none)
* IPv4: 10.7.19.102
* Trying 10.7.19.102:443...
* Connected to app.team1.test (10.7.19.102) port 443
* ALPN: curl offers h2,http/1.1
* (304) (OUT), TLS handshake, Client hello (1):
} [319 bytes data]
* CAfile: /etc/ssl/cert.pem
* CApath: none
* (304) (IN), TLS handshake, Server hello (2):
{ [122 bytes data]
* (304) (IN), TLS handshake, Unknown (8):
{ [47 bytes data]
* (304) (IN), TLS handshake, Certificate (11):
{ [1135 bytes data]
* (304) (IN), TLS handshake, CERT verify (15):
{ [264 bytes data]
* (304) (IN), TLS handshake, Finished (20):
{ [36 bytes data]
* (304) (OUT), TLS handshake, Finished (20):
} [36 bytes data]
* SSL connection using TLSv1.3 / AEAD-CHACHA20-POLY1305-SHA256 / [blank] / UNDEF
* ALPN: server accepted h2
* Server certificate:
* subject: CN=app.team1.test
* start date: Oct  4 13:11:02 2026 GMT
* expire date: Nov  5 13:11:02 2027 GMT
* subjectAltName: host "app.team1.test" matched cert's "app.team1.test"
```

```

* issuer: CN=team1 Local Root CA; O=CN Project team1
* SSL certificate verify ok.
* using HTTP/2
* [HTTP/2] [1] OPENED stream for https://app.team1.test/
* [HTTP/2] [1] [:method: GET]
* [HTTP/2] [1] [:scheme: https]
* [HTTP/2] [1] [:authority: app.team1.test]
* [HTTP/2] [1] [:path: /]
* [HTTP/2] [1] [user-agent: curl/8.7.1]
* [HTTP/2] [1] [accept: */*]
> GET / HTTP/2
> Host: app.team1.test
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/2 200
< server: nginx/1.31.6
< date: Sun, 04 Oct 2026 14:42:29 GMT
< content-type: text/html; charset=utf-8
< content-length: 177
< x-backend: A
< cache-control: no-store
< x-edge: app.team1.test
< x-upstream: 10.7.5.55:3001
<
{ [177 bytes data]
* Connection #0 to host app.team1.test left intact
<!doctype html><title>Backend A</title><h1>Served by Backend A</h1><p>host: Mayank-Thukrans-MacBook-Pro.local, port: 3001</p><p>Refresh to watch the load balancer alternate.</p>

```

Proofpoints: SSL connection using TLSv1.3 · subjectAltName: host "app.team1.test" matched · SSL certificate verify ok. · HTTP/2 200 · no -k, no IP in the URL.

7. Task F — HTTP caching

Response headers and conditional request

```
$ curl -sI https://app.team1.test/api/catalog
HTTP/2 200
server: nginx/1.31.6
date: Sun, 04 Oct 2026 14:50:38 GMT
content-type: application/json
content-length: 228
x-backend: A
cache-control: public, max-age=60
etag: "55df56e6b3ea9005"
last-modified: Wed, 01 Jan 2025 00:00:00 GMT
x-edge: app.team1.test
x-upstream: 10.7.5.55:3001

$ curl -sI https://app.team1.test/api/status
HTTP/2 200
server: nginx/1.31.6
date: Sun, 04 Oct 2026 14:50:37 GMT
content-type: application/json
content-length: 253
x-backend: B
cache-control: no-store
x-edge: app.team1.test
x-upstream: 10.7.16.171:3002

$ curl -sI -H 'If-None-Match: "55df56e6b3ea9005"' https://app.team1.test/api/catalog
HTTP/2 304
server: nginx/1.31.6
date: Sun, 04 Oct 2026 14:50:38 GMT
x-backend: A
cache-control: public, max-age=60
etag: "55df56e6b3ea9005"
last-modified: Wed, 01 Jan 2025 00:00:00 GMT
x-edge: app.team1.test
x-upstream: 10.7.5.55:3001
```

`/api/catalog` returns `Cache-Control: public, max-age=60`, an `ETag` and `Last-Modified`. `/api/status` is deliberately `no-store` because it shows live load-balancer data.

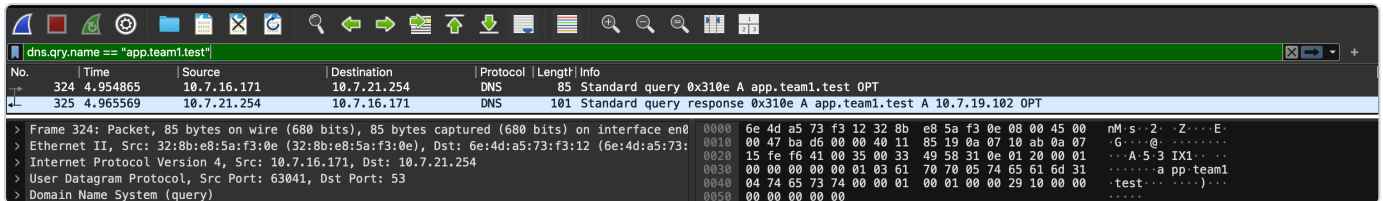
Case	What happens	Network cost
Fresh cache hit (within 60 s)	Browser reuses its stored copy; DevTools shows "(disk cache)"	No request at all
Conditional request (after 60 s)	Client sends <code>If-None-Match: "55df56e6b3ea9005"</code> ; server replies 304 Not Modified , headers only	One small round trip, no body
Full new request	No cache or ETag changed → 200 with full body and new ETag	Full body transferred

Both backends compute the same ETag from the content, so 304 works whichever backend the load balancer picks.

8. Task G — Wireshark protocol evidence

Captured on Mac 4 (client 10.7.16.171, interface en0) while running `curl https://app.team1.test/api/status`.

DNS



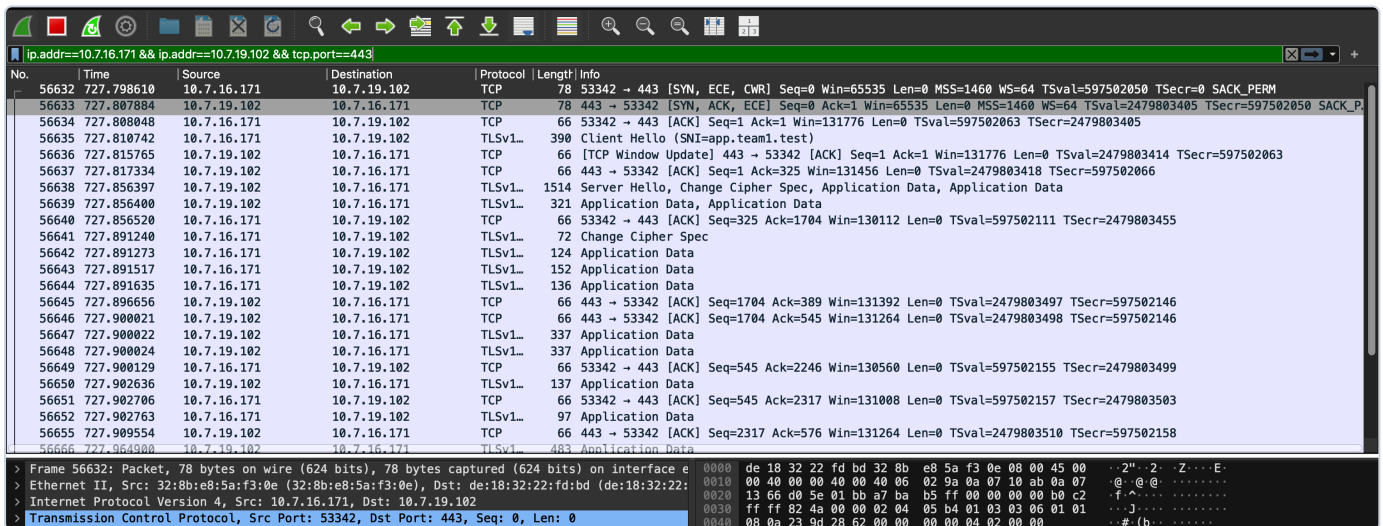
Wireshark capture of DNS traffic. The filter is `dns.qry.name == "app.team1.test"`. The capture shows two packets: a query (Frame 324) and a response (Frame 325).

No.	Time	Source	Destination	Protocol	Length	Info
324	4.954865	10.7.16.171	10.7.21.254	DNS	85	Standard query 0x310e A app.team1.test OPT
325	4.965569	10.7.21.254	10.7.16.171	DNS	101	Standard query response 0x310e A app.team1.test A 10.7.19.102 OPT

Frame 324: Packet, 85 bytes on wire (680 bits), 85 bytes captured (680 bits) on interface en0
> Ethernet II, Src: 32:8b:e8:5a:f3:0e (32:8b:e8:5a:f3:0e), Dst: 6e:4d:a5:73:f3:12 (6e:4d:a5:73:f3:12)
> Internet Protocol Version 4, Src: 10.7.16.171, Dst: 10.7.21.254
> User Datagram Protocol, Src Port: 63041, Dst Port: 53
> Domain Name System (query)

Filter `dns.qry.name == "app.team1.test"`. Frame 324: query A app.team1.test, 10.7.16.171:63041 → 10.7.21.254:53/UDP, ID 0x310e. Frame 325: response from our DNS, A 10.7.19.102, same ID. ≈10.7 ms.

TCP three-way handshake

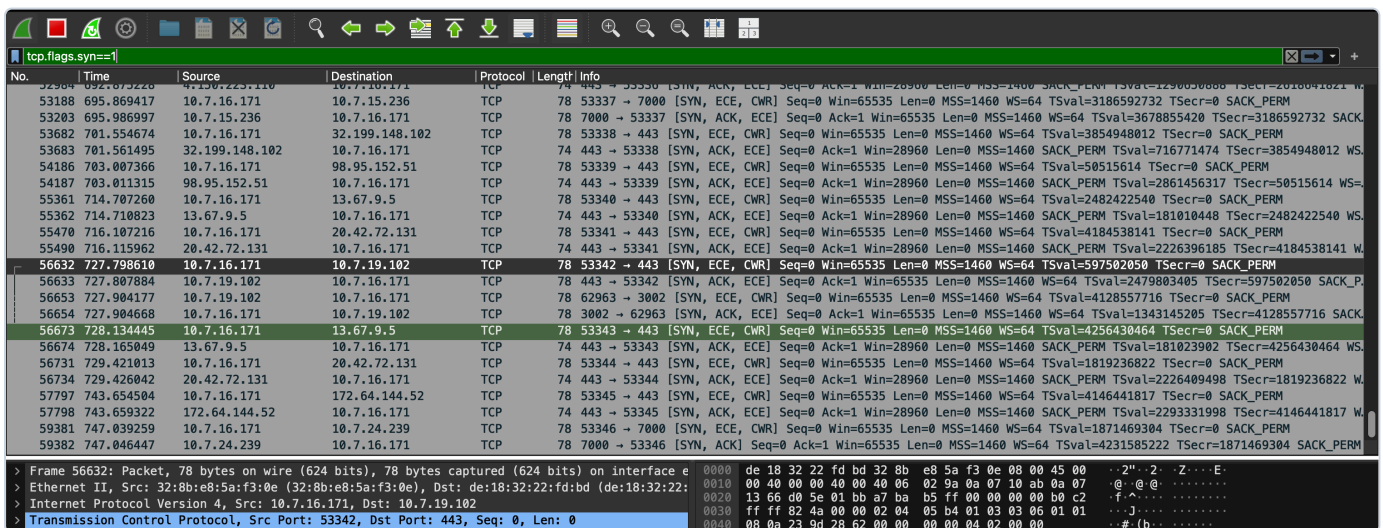


Wireshark capture of TCP traffic. The filter is `ip.addr==10.7.16.171 && ip.addr==10.7.19.102 && tcp.port==443`. The capture shows a three-way handshake between 10.7.16.171 and 10.7.19.102 on port 443.

No.	Time	Source	Destination	Protocol	Length	Info
56632	727.798610	10.7.16.171	10.7.19.102	TCP	78	53342 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56633	727.807884	10.7.19.102	10.7.16.171	TCP	78	443 → 53342 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56634	727.810742	10.7.16.171	10.7.19.102	TLSv1	390	Client Hello (SNI=app.team1.test)
56635	727.815765	10.7.19.102	10.7.16.171	TCP	66	[TCP Window Update] 443 → 53342 [ACK] Seq=1 Ack=1 Win=131776 Len=0 TSval=597502063 TSecr=2479803405
56636	727.817334	10.7.19.102	10.7.16.171	TCP	66	443 → 53342 [ACK] Seq=1 Ack=325 Win=131456 Len=0 TSval=597502066 TSecr=2479803418
56637	727.856397	10.7.19.102	10.7.16.171	TLSv1	1514	Server Hello, Change Cipher Spec, Application Data, Application Data
56638	727.856408	10.7.19.102	10.7.16.171	TLSv1	321	Application Data, Application Data
56639	727.856520	10.7.16.171	10.7.19.102	TCP	66	53342 → 443 [ACK] Seq=325 Ack=1704 Win=130112 Len=0 TSval=597502111 TSecr=2479803455
56640	727.891240	10.7.16.171	10.7.19.102	TLSv1	72	Change Cipher Spec
56641	727.891273	10.7.16.171	10.7.19.102	TLSv1	124	Application Data
56642	727.891517	10.7.16.171	10.7.19.102	TLSv1	152	Application Data
56643	727.891635	10.7.16.171	10.7.19.102	TLSv1	136	Application Data
56644	727.896656	10.7.19.102	10.7.16.171	TCP	66	443 → 53342 [ACK] Seq=1704 Ack=389 Win=131392 Len=0 TSval=2479803497 TSecr=597502146
56645	727.900021	10.7.19.102	10.7.16.171	TCP	66	443 → 53342 [ACK] Seq=1704 Ack=545 Win=131264 Len=0 TSval=2479803498 TSecr=597502146
56646	727.900022	10.7.19.102	10.7.16.171	TLSv1	337	Application Data
56647	727.900024	10.7.19.102	10.7.16.171	TLSv1	337	Application Data
56648	727.900129	10.7.19.102	10.7.16.171	TCP	66	53342 → 443 [ACK] Seq=545 Ack=2246 Win=130560 Len=0 TSval=597502155 TSecr=2479803499
56649	727.902636	10.7.19.102	10.7.16.171	TLSv1	137	Application Data
56650	727.902706	10.7.19.102	10.7.16.171	TCP	66	53342 → 443 [ACK] Seq=545 Ack=2317 Win=131008 Len=0 TSval=597502157 TSecr=2479803503
56651	727.902763	10.7.19.102	10.7.16.171	TLSv1	97	Application Data
56652	727.909554	10.7.19.102	10.7.16.171	TCP	66	443 → 53342 [ACK] Seq=2317 Ack=576 Win=131264 Len=0 TSval=2479803510 TSecr=597502158
56653	727.964900	10.7.19.102	10.7.16.171	TLSv1	483	Application Data

Frame 56632: Packet, 78 bytes on wire (624 bits), 78 bytes captured (624 bits) on interface en0
> Ethernet II, Src: 32:8b:e8:5a:f3:0e (32:8b:e8:5a:f3:0e), Dst: de:18:32:22:fd:bd (de:18:32:22:fd:bd)
> Internet Protocol Version 4, Src: 10.7.16.171, Dst: 10.7.19.102
> Transmission Control Protocol, Src Port: 53342, Dst Port: 443, Seq: 0, Len: 0

Filter `ip.addr==10.7.16.171 && ip.addr==10.7.19.102 && tcp.port==443`. 56632 SYN :53342→:443 Seq=0 MSS=1460 · 56633 SYN-ACK Seq=0 Ack=1 · 56634 ACK Seq=1 Ack=1 (≈9.4 ms) · only then 56635 Client Hello. Seq/Ack keep growing with the bytes sent (e.g. Ack=325, 1704) — TCP reliability in action.



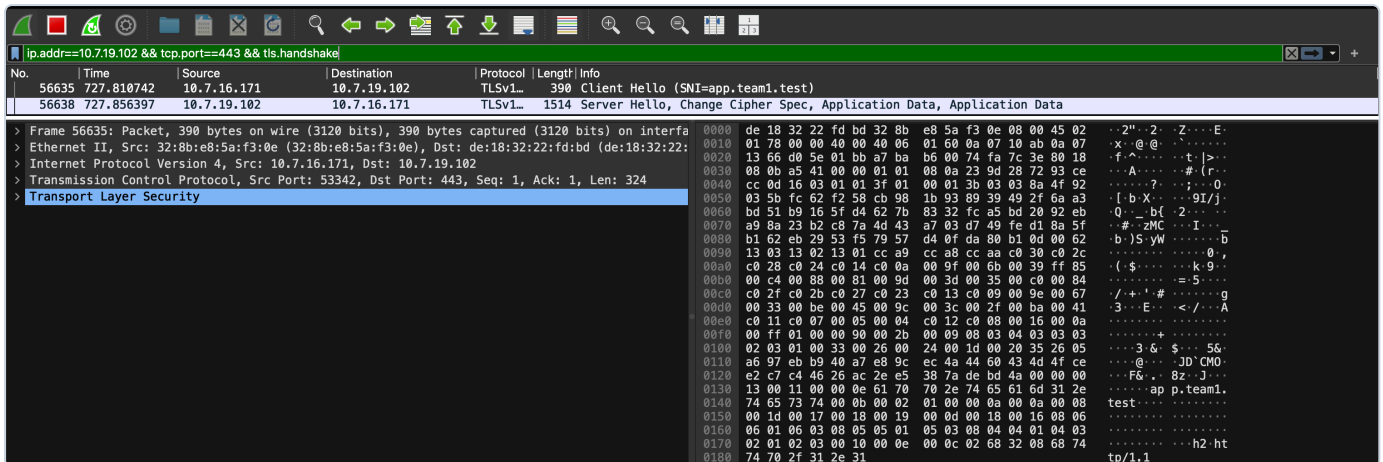
Wireshark capture of TCP traffic. The filter is `tcp.flags.syn==1`. The capture shows multiple TCP connections between 10.7.16.171 and 10.7.19.102 on port 443.

No.	Time	Source	Destination	Protocol	Length	Info
56632	727.798610	10.7.16.171	10.7.19.102	TCP	78	53342 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56633	727.807884	10.7.19.102	10.7.16.171	TCP	78	443 → 53342 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56634	727.810742	10.7.16.171	10.7.19.102	TCP	78	53342 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56635	727.815765	10.7.19.102	10.7.16.171	TCP	78	443 → 53342 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56636	727.817334	10.7.19.102	10.7.16.171	TCP	78	443 → 53342 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56637	727.856397	10.7.19.102	10.7.16.171	TCP	78	443 → 53342 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56638	727.856408	10.7.19.102	10.7.16.171	TCP	78	443 → 53342 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56639	727.856520	10.7.16.171	10.7.19.102	TCP	78	53342 → 443 [SYN, ACK, ECE] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56640	727.891240	10.7.16.171	10.7.19.102	TCP	78	53342 → 443 [SYN, ACK, ECE] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56641	727.891273	10.7.16.171	10.7.19.102	TCP	78	53342 → 443 [SYN, ACK, ECE] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56642	727.891517	10.7.16.171	10.7.19.102	TCP	78	53342 → 443 [SYN, ACK, ECE] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56643	727.891635	10.7.16.171	10.7.19.102	TCP	78	53342 → 443 [SYN, ACK, ECE] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56644	727.896656	10.7.19.102	10.7.16.171	TCP	78	443 → 53342 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56645	727.900021	10.7.19.102	10.7.16.171	TCP	78	443 → 53342 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56646	727.900022	10.7.19.102	10.7.16.171	TCP	78	443 → 53342 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56647	727.900024	10.7.19.102	10.7.16.171	TCP	78	443 → 53342 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56648	727.900129	10.7.19.102	10.7.16.171	TCP	78	443 → 53342 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56649	727.902636	10.7.19.102	10.7.16.171	TCP	78	443 → 53342 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56650	727.902706	10.7.19.102	10.7.16.171	TCP	78	443 → 53342 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56651	727.902763	10.7.19.102	10.7.16.171	TCP	78	443 → 53342 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56652	727.909554	10.7.19.102	10.7.16.171	TCP	78	443 → 53342 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM
56653	727.964900	10.7.19.102	10.7.16.171	TCP	78	443 → 53342 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 TSval=597502050 TSecr=0 SACK_PERM

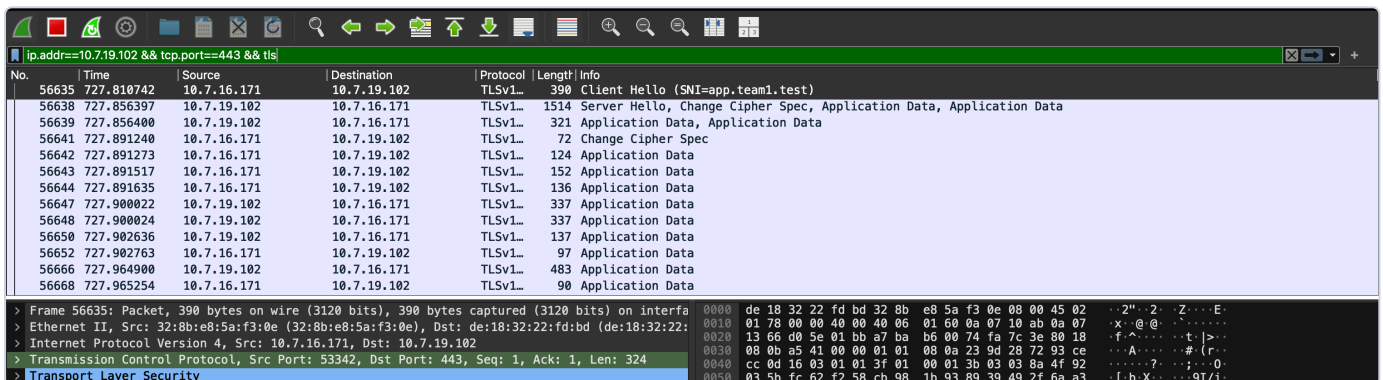
Frame 56632: Packet, 78 bytes on wire (624 bits), 78 bytes captured (624 bits) on interface en0
> Ethernet II, Src: 32:8b:e8:5a:f3:0e (32:8b:e8:5a:f3:0e), Dst: de:18:32:22:fd:bd (de:18:32:22:fd:bd)
> Internet Protocol Version 4, Src: 10.7.16.171, Dst: 10.7.19.102
> Transmission Control Protocol, Src Port: 53342, Dst Port: 443, Seq: 0, Len: 0

Filter `tcp.flags.syn==1`. Besides the client→edge handshake (56632/56633), frames 56653/56654 show nginx on 10.7.19.102 opening a separate TCP connection from port 62963 to Backend B 10.7.16.171:3002 — the second leg of the reverse proxy.

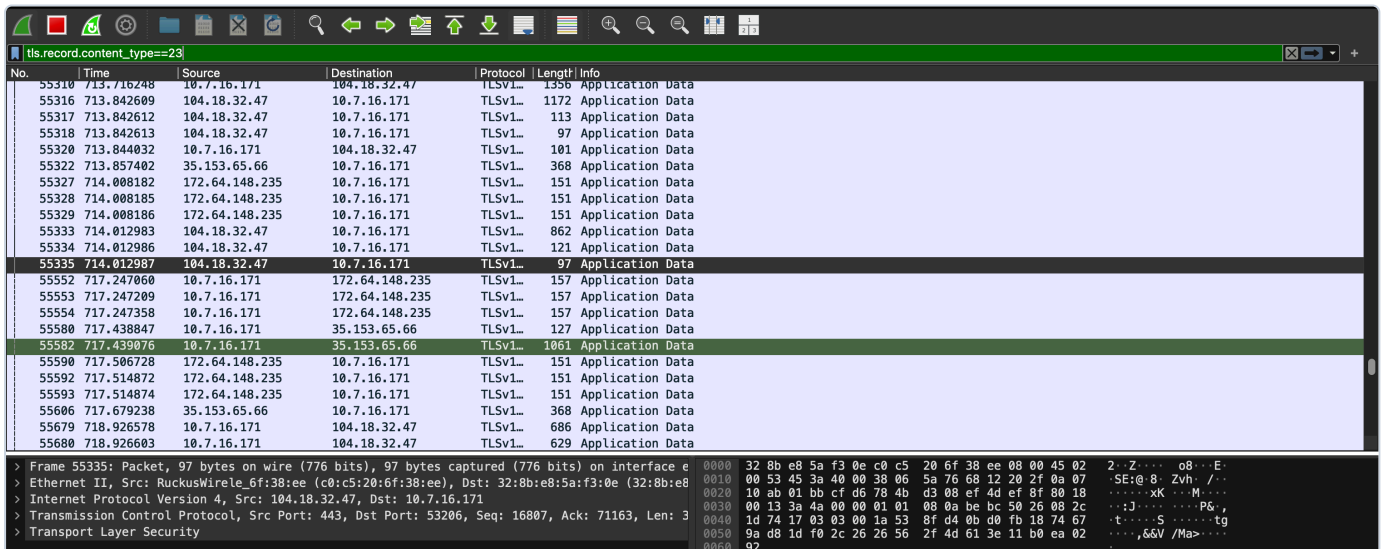
TLS handshake



Filter `ip.addr==10.7.19.102 && tcp.port==443 && tls.handshake`. 56635 Client Hello (SNI=app.team1.test); hex shows cipher suites 13 03 / 13 02 / 13 01 and ALPN h2, http/1.1). 56638 Server Hello + Change Cipher Spec + encrypted records.



Filter `... && tls`. After the handshake every record in both directions is only "Application Data". In TLS 1.3 the Certificate, CertificateVerify and Finished messages are already encrypted, so Wireshark cannot show them separately.



Filter `tls.record.content_type==23` (Application Data). The HTTP method, path, headers and JSON body are inside these encrypted records — not readable on the wire. Only IPs, ports, sizes and the SNI are visible.

Layer	Client port (ephemeral)	Server port (well-known)
DNS	63041/UDP	53/UDP on 10.7.21.254
HTTPS	53342/TCP	443/TCP on 10.7.19.102
Edge → backend	62963/TCP (nginx)	3002/TCP on 10.7.16.171

9. Failure demonstrations

9.1 Wrong destination port (L4)

Before – correct port 443

```
$ curl -v https://app.team1.test:443/api/status
* Connected to app.team1.test (10.7.19.102) port 443
* SSL certificate verify ok.
< HTTP/2 200
< x-backend: B
```

After – port 9999

```
$ curl -v https://app.team1.test:9999/api/status
* IPv4: 10.7.19.102
* Trying 10.7.19.102:9999...
* connect to 10.7.19.102 port 9999 from 10.7.19.102 port 63439 failed: Connection refused
* Failed to connect to app.team1.test port 9999 after 3 ms: Couldn't connect to server
curl: (7) Failed to connect to app.team1.test port 9999 after 3 ms: Couldn't connect to server

$ ping -c 3 app.team1.test
3 packets transmitted, 3 packets received, 0.0% packet loss

$ nc -vz 10.7.19.102 9999
nc: connectx to 10.7.19.102 port 9999 (tcp) failed: Connection refused
$ nc -vz 10.7.19.102 443
Connection to 10.7.19.102 port 443 [tcp/https] succeeded!
```

DNS resolves and ping works, but the host answers the SYN with a RST: nothing listens on 9999 → **Connection refused in 3 ms**. Ports and IPs are separate identifiers. Restored by using 443 again:

Restored

```
$ curl -s -D - -o /dev/null https://app.team1.test/api/status
HTTP/2 200
x-backend: A
```

9.2 Wrong DNS server on the client (L7 name resolution)

Observed on Mac 2 when its Wi-Fi DNS was reset to the default resolver.

Client using public DNS

```
$ networksetup -getdnsservers Wi-Fi
There aren't any DNS Servers set on Wi-Fi.      <- client falls back to router / 8.8.8.8

$ dig app.team1.test
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 61697
;; AUTHORITY SECTION:
.                86182    IN      SOA      a.root-servers.net. nstld.verisign-grs.com. 2026100400 1800 9
00 604800 86400
;; SERVER: 8.8.8.8#53(8.8.8.8)
;; WHEN: Sun Oct 04 20:12:03 IST 2026

$ curl -v https://app.team1.test/api/status
* Could not resolve host: app.team1.test
curl: (6) Could not resolve host: app.team1.test

$ ping -c 4 10.7.21.254 (and 10.7.5.55, 10.7.16.171)
4 packets transmitted, 4 packets received, 0.0% packet loss
```

Restored – client points at Mac 1 again

```
$ scripts/client-dns.sh primary
DNS for Wi-Fi: 10.7.21.254
```

```
in use: 10.7.21.254

$ dig app.team1.test
;; ANSWER SECTION:
app.team1.test.      30      IN      A       10.7.19.102
;; SERVER: 10.7.21.254#53(10.7.21.254)
```

Name lookup fails while IP connectivity is perfect — DNS and IP are independent layers.

9.3 One backend unavailable

Observed during testing: Wi-Fi packet loss made nginx's connection to Backend B time out. nginx marked B "temporarily disabled" and kept serving every request from A, then put B back after the fail timeout.

```
nginx error log and client view

# nginx error log on Mac 2 (Wi-Fi dropped packets to Mac 4)
[warn] upstream server temporarily disabled while connecting to upstream,
      upstream: "http://10.7.16.171:3002/api/status", host: "app.team1.test"
[error] upstream timed out (60: Operation timed out) while connecting to upstream

# the 6 client requests in that window – all served, all by A
x-backend: A
x-backend: A
x-backend: A
x-backend: A
x-backend: A
x-backend: A

# ~12 s later (fail_timeout over) – B is back in rotation
x-backend: A
x-backend: A
x-backend: B
x-backend: A
x-backend: B
x-backend: A
```

Still to record for the video: stopping Backend A deliberately (Ctrl+C on Mac 3) → only B; stopping both → 502 Bad Gateway ; wrong DNS record (scripts/dns-set-record.sh) → client reaches the wrong host. Commands are in Section 12.

10. Problems we hit and how we fixed them

Symptom	Layer	Root cause	Fix
<code>bind()</code> to <code>0.0.0.0:443</code> failed (48: Address already in use)	L4	nginx was already running; the install script's <code>pgrep -x nginx</code> missed it because nginx renames its process to "nginx: master process"	Changed the check to <code>pgrep -f 'nginx: master'</code> so the script reloads instead of starting a second copy
unable to get local issuer certificate on Mac 4	TLS	<code>make-certs.sh</code> was run twice → a new CA; Mac 4 still had the old <code>ca.crt</code> (same name, different key ID)	Sent the current <code>ca.crt</code> again and re-trusted it; never re-run <code>make-certs</code> after distributing the CA
Could not resolve host: <code>app.team1.test</code>	DNS	Client still using router / 8.8.8.8	<code>scripts/client-dns.sh</code> primary
<code>dig</code> to Mac 1 timed out from other Macs	DNS / L4	<code>dnsmasq</code> listening on localhost only	<code>listen-address=127.0.0.1,10.7.21.254</code> , restart <code>dnsmasq</code>
All requests to A for a few seconds	L3 / LB	Wi-Fi loss → nginx marked B down	Automatic recovery after <code>fail_timeout</code> ; use a stable hotspot for the demo
<code>zsh: unknown sort specifier</code>	shell	<code>zsh</code> treated <code>#</code> (optional) comments as glob patterns	<code>setopt interactivecomments</code>

11. What each component proves

- **DNS is a directory, not a connection** — it only returns 10.7.19.102; TCP to 443 is a separate step.
- **TCP before TLS before HTTP** — the capture shows SYN/SYN-ACK/ACK, then ClientHello, then encrypted data.
- **TLS termination at the edge** — client↔nginx is HTTPS/HTTP2; nginx↔backend is plain HTTP/1.1 on the LAN (the backends see `x_forwarded_proto: https`).
- **Load balancing** — one name, two servers; the client never knows backend IPs.
- **Single point of failure** — Mac 2 (edge) and Mac 1 (DNS) in Phase 1; Phase 2 adds a backup DNS on Mac 3 and a standby edge.

12. Video walkthrough script

Follows the evaluator's demonstration order. Run client commands on Mac 4 (or Mac 1). Keep each scene short; show the command and point at the highlighted line.

Scene 1 — Topology and IP inventory

Show: Section 1 of this report: topology diagram, IP table, request-flow diagram.

Say: We have four Macs on one Wi-Fi, subnet 10.7.0.0/19. Mac 1 is our private DNS, Mac 2 is the nginx edge that does TLS and load balancing, Mac 3 and Mac 4 are backend A and B on ports 3001 and 3002. A client only talks to the name app.team1.test; DNS turns it into Mac 2's IP and nginx forwards to a backend.

Scene 2 — All machines on the private LAN

Show: netinfo + pingall output.

```
commands
scripts/netinfo.sh
scripts/pingall.sh
```

Say: Each Mac shows its IP, mask /19, gateway 10.7.0.1 and interface en0. Pingall reaches all four machines — layer 3 connectivity is fine before we start any service.

Scene 3 — Resolve the private domain from a client

Show: dig answer and SERVER line; then dig @8.8.8.8.

```
commands
dig app.team1.test
dig @8.8.8.8 app.team1.test
```

Say: The answer is 10.7.19.102, Mac 2, and the SERVER line is 10.7.21.254 — our own dnsmasq, not Google. Asking 8.8.8.8 gives NXDOMAIN, which proves the name only exists inside our network.

Scene 4 — Open the service over HTTPS by name

Show: curl -v lines: TLSv1.3, subject, SAN matched, verify ok, HTTP/2 200. Also open https://app.team1.test in the browser and click the padlock.

```
commands
curl -v https://app.team1.test/api/status
```

Say: No -k and no IP address. TLS 1.3 is negotiated, the certificate's SAN matches app.team1.test and it is signed by our team CA, which we trusted on this Mac, so verification is OK and we get 200.

Scene 5 — Load balancing

Show: Alternating X-Backend values.

```
commands
for i in {1..6}; do curl -s -D - -o /dev/null https://app.team1.test/api/status | grep -i x-backend; done
```

Say: Same name, six requests, nginx round-robin sends them A, B, A, B. The client never knew the backend IPs — only the edge does.

Scene 6 — Wireshark: DNS, TCP, TLS

Show: Open the saved capture and apply the three filters from Section 8.

commands

```
dns.qry.name == "app.team1.test"
ip.addr==10.7.19.102 && tcp.port==443
ip.addr==10.7.19.102 && tcp.port==443 && tls
```

Say: First the DNS query on UDP 53 and the answer 10.7.19.102. Then the TCP handshake from ephemeral port 53342 to 443 — SYN, SYN-ACK, ACK with Seq 0 and Ack 1. Then the TLS ClientHello with SNI app.team1.test and the ServerHello. After that everything is Application Data — the HTTP request and response are encrypted, so Wireshark cannot read them.

Scene 7 — HTTP caching

Show: Cache-Control, ETag, then the 304.

commands

```
curl -sI https://app.team1.test/api/catalog
curl -sI -H 'If-None-Match: "55df56e6b3ea9005"' https://app.team1.test/api/catalog
```

Say: max-age=60 lets the client reuse the copy for 60 seconds without asking. After that it revalidates with the ETag; if nothing changed the server answers 304 Not Modified with no body. In the browser DevTools you see (disk cache) on reload.

Scene 8 — Fail one backend

Show: On Mac 3 press Ctrl+C on backend/run.sh A, run the loop, then start it again.

commands

```
# on Mac 3: Ctrl+C
for i in {1..6}; do curl -s -D - -o /dev/null https://app.team1.test/api/status | grep -i x-backend; done
# on Mac 3: backend/run.sh A -> loop again shows A and B
```

Say: With A stopped every request still succeeds, served by B, because nginx skips the failed upstream. When A comes back, round-robin resumes.

Scene 8b — Wrong port and wrong DNS (failure demos)

Show: Connection refused on 9999 while ping works; dig fails with an unreachable DNS server while ping by IP works.

commands

```
curl -v https://app.team1.test:9999/api/status
ping -c 3 app.team1.test
scripts/client-dns.sh bogus ; dig app.team1.test ; ping -c 3 10.7.19.102
scripts/client-dns.sh primary
```

Say: Port 9999: the host is up but no service listens, so TCP is refused — a layer-4 problem. Bad DNS: names fail but ping by IP works — DNS and IP are independent. We restore by using 443 and pointing DNS back at Mac 1.

Scene 9 — Phase 2 resilience (when done)

Show: Backup DNS on Mac 3, TTL 30 s, DNS cutover.

```
commands
```

```
scripts/client-dns.sh both  
scripts/ttl-watch.sh
```

Say: Clients list Mac 1 and Mac 3 as DNS; stopping dnsmasq on Mac 1 still resolves via Mac 3. With TTL 30 the cached answer lasts up to 30 seconds after a record change.

Scene 10 — Diagnose an injected fault

Show: Work through layers out loud.

```
commands
```

```
dig app.team1.test          # DNS ok?  
nc -vz 10.7.19.102 443      # TCP ok?  
curl -v https://app.team1.test # TLS ok? HTTP status?  
tail /tmp/nginx-team-error.log # backend side
```

Say: We always go bottom-up along the request: name resolution, then TCP to the port, then the TLS handshake, then the HTTP status and the nginx log for the backend leg.

Closing line

One request touches every layer: DNS on UDP 53 finds the edge, TCP on 443 builds a reliable connection, TLS 1.3 encrypts it with a certificate from our own CA, nginx terminates TLS and load-balances over HTTP to two backends, and caching headers decide whether the request needs to happen at all.